# INFORME DE AVANCE 02

# Sistema de Gestión de Pedidos
# Tienda de Tecnología


Facultad de Ingeniería

Carrera Profesional de Ingeniería de Sistemas e Informática


**Asignatura:** Herramientas de Desarrollo

**Grupo:** 05


**Integrantes**

Jara Moya, Eduardo Jesús — U21302190

Galvan Zamudio, Lisbert Wiliams — U22217814

Velasquez Pacheco, Nery Emerson — U23272063

Rivera Hipolito, Carlos Fernando — U23216154

Reyes Vicente, Bruno Santiago — U19221035


Repositorio: github.com/Eduardo-Jara03/Proyecto-pedidos-tiendatecnologia-grupo05

**Asignatura:** Herramientas de Desarrollo

**Carrera:** Ingeniería de Sistemas e Informática

**Grupo:** 05

**Proyecto:** Sistema de pedidos para una tienda de tecnología

**Repositorio:** https://github.com/Eduardo-Jara03/Proyecto-pedidos-tiendatecnologia-grupo05

## 1. Resumen ejecutivo

El proyecto desarrolla una solución web para registrar y administrar clientes, productos y pedidos de una tienda de tecnología. La entrega contempla frontend web, API REST con Express, persistencia SQLite y pruebas automatizadas. El trabajo colaborativo se organiza mediante Git/GitHub, ramas por funcionalidad, commits descriptivos, Pull Requests y resolución de conflictos documentada.

Esta copia recuperada incluye el frontend y la documentación disponible, el backend de clientes/productos, el esquema relacional, la ruta de pedidos recuperada y montada en Express, además de pruebas ampliadas. El código se verificó con 30 pruebas aprobadas en el entorno de verificación disponible.

## 2. Equipo de trabajo

| Integrante | Código |
|---|---|
| Jara Moya, Eduardo Jesús | U21302190 |
| Galvan Zamudio, Lisbert Wiliams | U22217814 |
| Velasquez Pacheco, Nery Emerson | U23272063 |
| Rivera Hipolito, Carlos Fernando | U23216154 |
| Reyes Vicente, Bruno Santiago | U19221035 |

Las responsabilidades finales deben contrastarse con las tarjetas del tablero y los commits/PR reales; el autor de un commit no demuestra por sí solo la asignación de una actividad ni una revisión formal.

## 3. Problema y solución

La tienda necesita centralizar el registro de clientes, el catálogo de productos y los pedidos asociados. La solución presenta una interfaz TechStore y una API que valida las operaciones y conserva los datos relacionales en SQLite.

### *Componentes técnicos*

- **Frontend:** HTML, CSS y JavaScript.
- **Backend:** Node.js y Express.
- **Persistencia:** SQLite con `better-sqlite3`, claves foráneas, restricciones e índices.
- **Versionamiento:** Git y GitHub.
- **Pruebas:** `node:test` y peticiones HTTP a una instancia temporal de la API.

## 4. Requerimientos funcionales

| ID | Requerimiento | Criterio verificable |
|---|---|---|
| RF-01 | Gestionar clientes | Registrar, consultar, buscar y actualizar; validar campos, correo y documento único |
| RF-02 | Gestionar productos | Registrar, consultar, buscar y actualizar; validar código único y precio positivo; desactivar lógicamente |
| RF-03 | Registrar pedidos | Asociar cliente y uno o varios productos activos con cantidades positivas |

| ID | Requerimiento | Criterio verificable |
|---|---|---|
| RF-04 | Calcular importes | Calcular subtotal y total en backend usando el precio guardado en SQLite |
| RF-05 | Consultar pedidos | Listar, buscar y obtener cabecera y detalles |
| RF-06 | Mantener integridad | Usar claves foráneas, restricciones y transacciones para pedido y detalles |
| RF-07 | Gestionar estado | Cancelar pedido y evitar una segunda cancelación |
| RF-08 | Verificar el sistema | Exponer `/api/health` y ejecutar pruebas automatizadas |

## 5. Actividades y trazabilidad

La relación entre requerimientos, actividades, responsables y evidencia se detalla en `matriz-trazabilidad.md` (matriz-trazabilidad.md). La secuencia de control definida es:

**Requisito → actividad → responsable → rama/commit → revisión del PR → integración → evidencia.**

Una actividad solo se marca como terminada cuando el resultado puede verificarse. Las asignaciones que no estén demostradas por el tablero deben confirmarse con el equipo.

## 6. Organización del repositorio y estrategia de ramificación

Se trabaja con una rama estable (`main`), una rama de integración (`develop`) y ramas temáticas para funcionalidades y documentación. Las ramas identificadas en el historial incluyen `feature/clientes`, `feature/productos`, `feature/pedidos`, `feature/backend-api`, `docs/flujo-git` y `docs/matriz-trazabilidad`.

### *Flujo de trabajo*

1. Actualizar la rama base.
2. Crear una rama para una tarea concreta.
3. Implementar cambios pequeños y ejecutar pruebas.
4. Registrar commits descriptivos.
5. Publicar la rama y abrir un Pull Request hacia `develop`.
6. Revisar cambios y resolver observaciones.
7. Integrar mediante merge solo después de la revisión.
8. Comprobar el estado final y registrar la evidencia.

Los PR e historiales de merge existentes son evidencia de colaboración e integración. La existencia de un PR no prueba que haya recibido una aprobación formal; debe adjuntarse la vista real de Reviews cuando exista.

## 7. Gestión de actividades y proyecto

El tablero del equipo debe usar los cinco estados solicitados:

1. **Pendiente:** tarea identificada.
2. **Seleccionado:** tarea priorizada para iniciar.
3. **En progreso:** tarea en ejecución.
4. **En revisión:** tarea enviada a revisión.
5. **Hecho:** tarea revisada, integrada y con evidencia.

Cada tarjeta debe indicar requisito, descripción, responsable, prioridad, rama, PR/commit y evidencia de cierre. El tablero real y sus responsables no se pueden reconstruir con certeza solo desde el código: deben adjuntarse capturas o un enlace accesible.

## 8. Conflictos de fusión

El archivo `resolucion-conflictos.md` (resolucion-conflictos.md) documenta tres conflictos relacionados con `README.md`: entre `develop` y `feature/pedidos`, durante la integración de `feature/pedidos` a `develop`, y al integrar `develop` a `main`. El documento identifica causas, archivo afectado y resolución registrada.

Para la entrega, adjuntar capturas de las marcas de conflicto y del merge final en Git/GitHub. La descripción escrita no sustituye las capturas originales.

## 9. Incidencias, solución y prevención

El archivo `incidencias.md` (incidencias.md) registra tres incidencias técnicas: bloqueo de permisos en una unidad FAT32, fallo de instalación de `better-sqlite3` por Python no ejecutable y ausencia del montaje `/api/pedidos` en Express. Incluye impacto, causa, contención, corrección y medidas preventivas.

## 10. Pruebas y verificación

Se ejecutó la suite automatizada con estos resultados:

| Grupo de pruebas | Casos aprobados |
|---|---|
| Salud, clientes, productos y validaciones | 18 |
| Pedidos, persistencia, edición, cancelación y eliminación | 12 |
| **Total** | **30 aprobados, 0 fallidos** |

Comandos de reproducción:

```
npm.cmd install
npm.cmd test
npm.cmd start
```

Las pruebas usan una base de datos temporal y no dependen de los datos de la base local. La instalación debe verificarse en el equipo que realiza la entrega, porque `better-sqlite3` puede requerir un binario precompilado compatible o herramientas de compilación.

## 11. Alcance de la versión 1 y trabajo pendiente

### *Incluido en esta copia recuperada*

- Interfaz de clientes, productos y pedidos.
- API REST para clientes, productos y pedidos.
- Esquema SQLite y restricciones de integridad.
- Validación de datos, búsqueda, edición, cancelación y eliminación de pedidos.
- Pruebas automatizadas y documentación de endpoints.
- Matriz de trazabilidad, registro de incidencias y documentación de conflictos.

### *Pendiente de evidencia o confirmación externa*

- Capturas del tablero real con los cinco estados y tareas asignadas.
- Evidencia de colaboración y participación de cada integrante.
- Capturas/enlaces de Reviews reales de los PR; no se deben inventar aprobaciones.
- Capturas del conflicto en el momento de producirse y de su resolución.
- Confirmación del PR final del backend y de la integración efectiva en `develop`.
- Captura de la ejecución de las 30 pruebas en el equipo de entrega y evidencia de la exposición.

## 12. Conclusiones

La solución recuperada reúne la interfaz, la API y la persistencia relacional, y amplía las pruebas para cubrir pedidos además de clientes y productos. El uso de transacciones e integridad referencial reduce el riesgo de registrar pedidos incompletos. La trazabilidad y el historial de Git ayudan a relacionar cambios y documentación con las actividades del equipo.

Antes de entregar, el equipo debe validar el entorno de ejecución y adjuntar las evidencias externas indicadas. No deben marcarse como revisados, aprobados o integrados aquellos cambios cuyo estado real no se haya comprobado en GitHub.